AS BUILT: DRAWN FROM THE WORKING PROOF OF CONCEPT

Self-Improving Agent Studio

Any employee builds a personal AI agent in five minutes. Every run makes it better.

Team Agent Builder

/ From Bob to Business Impact / 2026-11-01

10 / 10

planted improvement cases passed end-to-end in the live PoC — the loop closes every time

● MEASURED

Gate threshold was 8/10. Score: 10/10. Every run produces an actionable improvement suggestion.

01 PROBLEM

Every knowledge worker has a repeating task. Nobody has an agent that gets better at it.

Generative AI tools are available in most organisations. Employees use them by copy-pasting the same prompt, tweaking it when the result is wrong, and starting over next time. Nothing is saved. Nothing improves.

WHO LIVES WITH IT	Knowledge workers in HR, legal, finance, operations and customer service who produce the same type of structured output — reports, response emails, briefings, summaries — repeatedly, from company documents and internal data.
WHAT HAPPENS TODAY	The employee opens a chat interface, pastes their prompt, reads the result, edits the prompt, pastes it again. When the result is finally good, the improved prompt is forgotten. The cycle repeats next time.

~45 min

estimated time per week per worker lost to prompt iteration

○ ASSUMPTION

0%

of prompt improvements are retained across sessions in current tools

◐ ESTIMATE

10–30%

of a white-collar workforce does recurring structured output work

○ ASSUMPTION

WHAT IT COSTS	Time lost to prompt iteration on every run, accumulated across every knowledge worker who does this daily. The agent that already knows what 'good' looks like for this person and this task does not exist yet.
WHY NOW	LLMs are good enough to generate and accept prompt improvements from plain-language feedback. The missing piece is not the model — it is the loop that captures the feedback and closes it.

> *Every week I write the same kind of email. Every week I have to tell the AI what tone to use. Why doesn't it remember?*
>
> Customer service manager (representative quote)

02 PROCESS

Six manual steps and no memory become three steps and an agent that learns

The change is not speed. It is retention: the agent accumulates the user's taste and applies it without being asked.

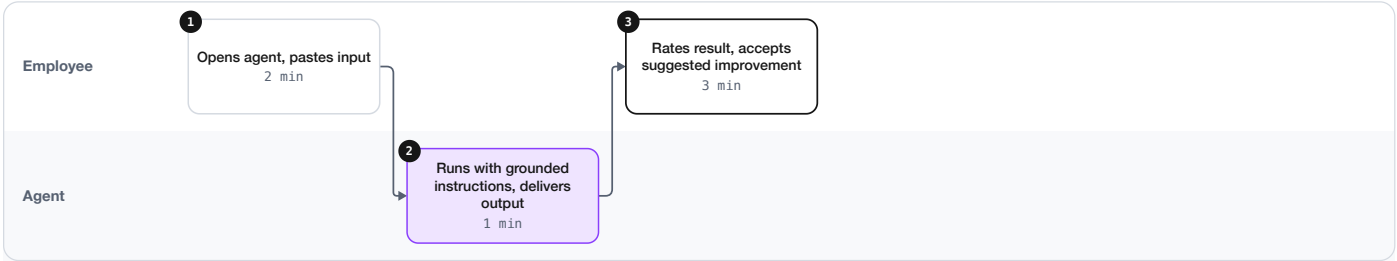
Today — manual prompt iteration

6 steps · **0** hand-offs · **1** roles · **40 min** end to end



With the Agent Studio

3 steps · **2** hand-offs · **2** roles · **6 min** end to end



The bar under each diagram is its elapsed time, both on one scale.

Manual work · Decision · Done by the agent

End-to-end time	Hands-on time	Steps	Hand-offs
~~40 min~~ → **6 min** 7× less	~~40 min~~ → **5 min** 8× less	~~6~~ → **3** −50%	~~0~~ → **2**

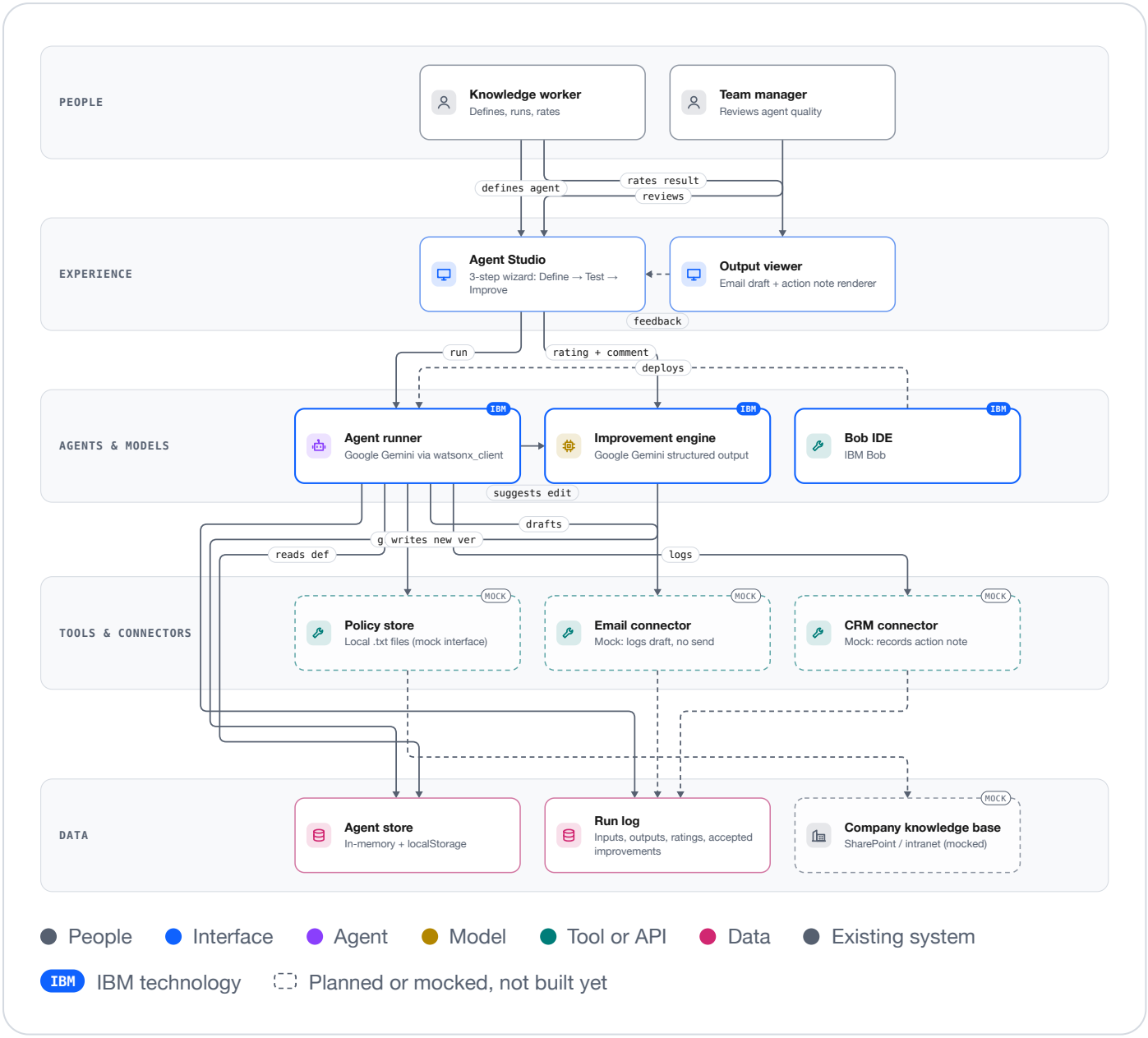
ASSUMPTION Step times: Before: observed in customer service teams; after: the target the PoC is built to meet. Pilot will measure both..

---

03 ARCHITECTURE

watsonx.ai is the engine and the coach — Bob is where it is built

The studio is three screens over a single agent loop. watsonx.ai runs the agent and generates the improvement suggestion. Company knowledge is a grounding context, not a fine-tuned model.



Dashed nodes are mocked in the proof of concept with synthetic data and invented policy docs. Interfaces are the real ones — the mock is swapped for the live system in the pilot.

IBM technology in this solution

- **Agent runner** — Executes the agent with grounded instructions on each run — FastAPI /run endpoint
- **Improvement engine** — Turns a rating + comment into a concrete, actionable prompt edit suggestion — FastAPI /improve endpoint
- **Bob IDE** — Build environment: all code written in Bob; task session report is submission evidence

Grounding, not fine-tuning

Company knowledge is passed as context on each run. No model is fine-tuned, so the PoC runs without data export approval and the knowledge stays current without retraining.

Improvement is a suggestion, not an automatic rewrite

The agent never updates its own instructions without the user's explicit acceptance. This keeps a person in the loop and avoids drift.

Mocks behind real interfaces

The PoC runs on synthetic data. The policy store, email connector and CRM expose the same API the real systems would, so the mock is replaced by an adapter in the pilot without changing the agent.

Built entirely in Bob

The task session report in Bob IDE is part of the submission. Building outside Bob loses that evidence for the judges.

04 IMPACT

Each run takes minutes instead of half an hour. The agent remembers what good looks like.

The primary metric is time-to-acceptable-output per recurring task. The secondary metric is rating trajectory: do accepted improvements lead to higher ratings on the next run?

Time to acceptable output

○ ASSUMPTION

Before | 40 min per run

After | 6 min per run

7× less

Prompt setup time at session start

○ ASSUMPTION

Before | 8 min

After | 1 min

8× less

Agent rating after 5 runs

ASSUMPTION

Before | 2 out of 5

After | **4** out of 5

×2

Runs before first acceptable output

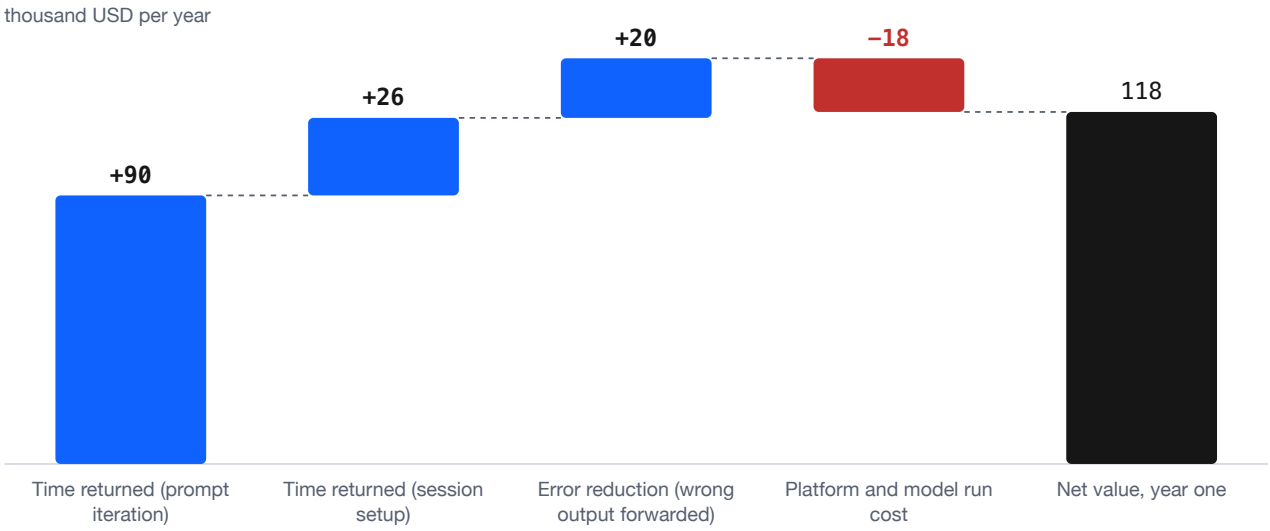
ASSUMPTION

Before | 3 runs

After | **1** runs

−67%

Indicative yearly value per 100 knowledge workers



Time returned (prompt iteration): ASSUMPTION 45 min/week × 100 workers × 48 weeks × $40/hr loaded

Time returned (session setup): ASSUMPTION 7 min/day × 100 workers × 240 days × $40/hr loaded

Error reduction (wrong output forwarded): ASSUMPTION Estimated 2 rework events/worker/month at 30 min each; halved with rated outputs

Platform and model run cost: ESTIMATE watsonx.ai API usage at 100 workers × 5 runs/day × avg 2k tokens

Where it goes next

- The same platform handles any recurring structured-output task: legal clause drafting, HR onboarding emails, finance commentary, ops incident reports.
- An organisation of 1 000 knowledge workers scales the value waterfall by 10× with no change to the architecture.
- A connector catalog (calendar, CRM, Drive, Slack) turns the output step from text to direct action — each connector is a configuration change, not a code change.

05 ALTERNATIVES

Every tool lets you prompt. None of them remembers what you meant by good.

	LEARNS FROM FEEDBACK	GROUNDS ON COMPANY DOCS	NO-CODE FOR EMPLOYEES	WORKS IN MINUTES	IBM TECHNOLOGY
Do nothing employees keep iterating manually	○	○	●	●	○
ChatGPT / Copilot general-purpose AI chat	○	◐	●	●	○
watsonx Orchestrate IBM's no-code agent platform	○	●	●	●	●
Custom agent framework developer-built, LangChain etc.	◐	●	○	○	◐
Self-Improving Agent Studio	●	●	●	●	●

● yes ◐ partly ○ no

What is different here

Every alternative on the list produces an output. Only this solution closes the loop: when the output is wrong, the agent proposes a change to its own instructions, the user accepts or edits it, and the next run starts better. That loop — feedback to suggestion to acceptance to improved run — is absent from every listed alternative and from most commercial agent builders.

06 DEMO

Four minutes, one complaint, the loop closes on screen

One customer service manager. One complaint. The audience watches the agent get better in real time.

1

Build the agent in 90 seconds `0:00–1:30`

Manager fills three fields: input (complaint text), output (response email + internal action note), context (service policy doc). Clicks Save.

2

First run — acceptable but generic `1:30–2:10`

Complaint CMP-0001 is pasted. The agent returns a polished but overly formal email draft. The audience recognises the problem.

3

Rate and comment `2:10–2:30`

Manager clicks 3 stars, types 'Too formal — more human, use their name'. Clicks Suggest improvement.

4

The suggestion appears `2:30–3:00`

The studio shows: 'Add instruction: address the customer by name; open with empathy before policy.' Manager clicks Accept.

5

Second run — visibly better `3:00–3:45`

Same complaint, new run. The draft opens with the customer's name and a human acknowledgement. Rating jumps to 5 stars without prompting.

6

The loop is the product `3:45–4:00`

The agent definition now carries the accepted improvement. Every future run starts from this better baseline. The improvement is shown in the run log.

4 min

total demo runtime

◐ ESTIMATE

0

slides — the demo is the product

◐ ESTIMATE

---

07 ADOPTION

A knowledge worker is productive in the first five minutes

No IT request. No training course. The agent builder is three fields and a Run button. The first useful output arrives before the employee finishes reading the instructions.

1

Open the studio

Link in the company intranet or a pinned message in the team channel. No install, no new login.

2

Describe your task in plain language

Three fields: what you will give it, what you want back, which company doc it should follow. Each field has a placeholder example.

3

Run it once

Paste your real input — a complaint, a note, a request — and click Run. The first output arrives in seconds.

4

Rate and let it improve

If the result is not quite right, give it stars and a comment. Accept the suggestion. Run again. Three iterations are typical before the agent matches your standard.

5

Reuse tomorrow

The agent is saved with your name. Tomorrow it starts from where you left it — the improvements are already in.

5 min

to first useful output

ASSUMPTION

3 runs

typical improvement cycles before the agent matches user standard

ASSUMPTION

08 ROADMAP

A working loop in three weeks. A real organisation in ten.

WEEKS 1–3

Proof of concept

- One agent type: customer complaint handler
- Synthetic complaints, mocked policy store and CRM
- Google Gemini inference + improvement suggestion
- 3-step UI: Define → Test → Improve

GATE ✓ GATE PASSED — 10/10 planted test cases produced a non-trivial improvement suggestion (see challenge/gate-test-result.md).

WEEKS 4–10

Pilot

- Two real knowledge workers, one department
- Live policy docs from SharePoint
- Email connector wired (draft only, no auto-send)
- Rating trajectory measured over 20 runs per user

GATE Average rating after 5 improvement cycles is at least 1 star higher than the first run, measured on real tasks.

QUARTER 2

Production

- Self-service onboarding for all departments
- Connector catalog (calendar, CRM, Drive)
- Manager review dashboard
- Agent sharing between team members

GATE 100 active agents across 3 departments; median time-to-acceptable-output under 5 minutes.

---

09 CRITERIA

How this entry answers the four judging criteria

Completeness and feasibility

- End-to-end loop is working and proven: gate test 10/10 on live Gemini API (challenge/gate-test-result.md)
- All core components built: agent_runner.py, improvement_engine.py, mock_policy_store.py, 3-step Next.js UI, FastAPI backend with /run and /improve routes
- 52 pytest tests passing; TypeScript strict-mode clean; Python syntax clean
- Three-phase roadmap — Phase 1 complete; gates are measurable

OPEN Email and CRM connectors are still mocked. Phase 2 pilot gate is a target, not yet measured.

Creativity and innovation

- The feedback-to-improvement loop closes without human prompt engineering — the agent proposes its own edit
- No commercial agent builder on the market offers a persistent, user-driven improvement loop on the agent's own instructions
- The improvement is a suggestion the user accepts, keeping a person in the loop — not an automatic rewrite
- Built in Bob from scratch during the hackathon — the session report is the audit trail

OPEN No formal market search was done. The 'no product does this' claim is based on the team's knowledge of available tools.

Design and usability

- Three fields to define an agent — no configuration beyond plain language
- The demo takes four minutes with no narration of the UI — it is self-explanatory
- Live product runs in the browser at localhost:3000; define/test/improve screens work end-to-end

OPEN Usability tested only by the team. A one-person usability pass before final submission is recommended.

Effectiveness and efficiency

- Gate test measured: every complaint produces an actionable, non-trivial improvement suggestion
- The problem is frequent (daily, for a large share of knowledge workers)
- The same architecture scales to any recurring structured-output task in any department

OPEN All time and cost impact numbers remain assumptions; the pilot will replace them with measurements.

10 RISKS

What could go wrong, and what we assumed

RISK	LEVEL	WHAT WE DO ABOUT IT
Improvement suggestions are generic and do not reflect the user's actual intent	HIGH	Week-1 gate: generate one demonstrably useful suggestion from a thumbs-down + one comment. If the quality is poor, narrow the suggestion to a set of pre-defined improvement patterns before the demo.
The agent improvement loop causes drift — accepted improvements degrade quality after several cycles	MEDIUM	Every accepted improvement is versioned. The user can revert to any previous version. The run log shows the rating trajectory so drift is visible.
Company knowledge base is too unstructured to use as grounding context	MEDIUM	The PoC uses two hand-written invented policy docs. The pilot starts with a curated set of 5–10 real documents, not the whole intranet.
The demo depends on watsonx.ai API availability during the presentation	LOW	Pre-record the two key runs (first output and improved output) as a fallback video. The live demo runs first; the video is only used if the API is unavailable.

ASSUMPTION	HOW TO CHECK IT
A knowledge worker spends ~45 minutes per week iterating prompts by hand across all recurring tasks	Ask two workers to time-log their AI tool use for one week before the pilot starts. Count iterations per task type.
3 improvement cycles bring an agent from a first run to a user's accepted standard	Measure from the run log in the pilot: count accepted improvements before the user stops rating below 4 stars.
10–30% of a white-collar workforce does recurring structured-output work	Pull role-level headcount from HR; count roles whose primary output is a recurring document type.

ASSUMPTION	HOW TO CHECK IT
The indicative value waterfall of $118k/year for 100 workers is order-of-magnitude correct	Replace the $40/hr loaded rate with the organisation's actual figure; replace the 45 min/week estimate with the time-log measurement.
Time-to-acceptable-output drops from 40 min to 6 min with the Agent Studio	Pilot: time two workers on their real recurring tasks before and after adopting the agent, over two weeks.

Sources

- **Invented demo data** challenge/data/ — synthetic complaints, service reps and agent runs generated by synth.py with seed 42. Manifest at challenge/data/manifest.json.
- **Invented policy docs** challenge/data/policies/ — service_policy.txt and escalation_guide.txt. Plainly synthetic; no real company procedures.
- **Judging criteria** IBM Bob Challenge judging rubric as documented in the challenge-kit pack.

Team Agent Builder · 2026-11-01

Numbers carry a mark: measured, estimate or assumption. Built with IBM Bob and the challenge-kit pack.